

CPT102 Data Structures

Weeks 1 and 2 practice exercises – solutions

Erick Purwanto, Thomas Selig, Kok Hoe Wong

March 30, 2026

This document contains solutions to the problems in the exercise sheet. Note that some of the solutions are not fully developed. For example, the algorithm analysis is generally done very quickly, and some of the explanations are just sketched.

Exercise 1 Basic array-based algorithms

This warm-up exercise implements two straightforward array-based algorithms.

* 1. (*Array average*)

- a) Write an algorithm `CALCULATEAVERAGE(arr)` that takes as input an array of numbers, and returns the average (mean) of its elements.
- b) What are the time and space complexities of this algorithm?

** 2. (*Finding duplicates*)

- a) Write an algorithm `CONTAINS DUPLICATES(arr)` that takes as input an array, and returns **true** if the array contains duplicate values, and **false** otherwise.
- b) What are the time and space complexities of this algorithm?

Answer of exercise 1

1. (*Array average*)

- a) This is a straightforward algorithm that you will likely already be familiar with.

Procedure `CALCULATEAVERAGE(arr)`

$n \leftarrow \text{len}(\text{arr})$, $\text{sum} \leftarrow 0$;

for $i \leftarrow 0$ **to** $n - 1$ **do**

$\text{sum} \leftarrow \text{sum} + \text{arr}[i]$;

return sum/n ;

- b) The algorithm uses a loop with n iterations, so time complexity is $O(n)$ (linear). Additional memory is only simple variables, so space complexity is $O(1)$ (constant).

2. (Finding duplicates)

Procedure CONTAINS DUPLICATES(**arr**)

$n \leftarrow \text{len}(\mathbf{arr})$;

for $i \leftarrow 0$ **to** $n - 1$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**
 if $\mathbf{arr}[i] = \mathbf{arr}[j]$ **then**
 return true;

return false;

The algorithm uses two nested for loops. The outer loop has n iterations, while the inner one has $(n-1-i)$ iterations. The total time is therefore $\sim \sum_{i=0}^{n-1} (n-1-i) = \sum_{j=0}^{n-1} j = \frac{n(n-1)}{2} = O(n^2)$. Thus the algorithm has *quadratic* time complexity. It has constant space complexity as the additional memory is limited to elementary operations.

Exercise 2 Counting distinct values

In this exercise, we will work on algorithms COUNTDISTINCTVALUES(**arr**) that take as input an array **arr**, and return the number of *distinct* values in **arr**. For example, the array $[1, 5, -1, 5, 2, 5, 1, 7]$ contains 5 distinct values: $-1, 1, 2, 5, 7$.

1. (General implementation)

In this question, we will work on a general implementation that solves the problem for any array input. Note that there exist more efficient solutions using other data structures, or sorting algorithms, but for now we limit ourselves to the tools learned in the first two weeks of the course. The principle is as follows. Let REMOVE(**arr**, i) be a procedure that, given an array **arr** and an index i , sets all values $\mathbf{arr}[j]$ such that $j > i$ and $\mathbf{arr}[j] = \mathbf{arr}[i]$ to some “sentinel” value such as **null**.¹

*** a) Using the procedure REMOVE as a helper algorithm (i.e., without writing it for now), implement the desired algorithm COUNTDISTINCTVALUES(**arr**).

* b) Now complete the algorithm by implementing the REMOVE procedure.

** c) What are the time and space complexities of this algorithm?

2. (Sorted case)

In this question, we consider the case where **arr** is an array of numbers which is *sorted* (in increasing order), i.e. we have: $\mathbf{arr}[0] \leq \mathbf{arr}[1] \leq \dots \leq \mathbf{arr}[n-1]$.

** a) Using this fact, improve the algorithm COUNTDISTINCTVALUES so that it runs in linear time.

* b) What is the space complexity of this new algorithm?

*** c) We will see in Week 6 that it is possible to sort an array of numbers in log-linear $O(n \cdot \log n)$ time, and constant memory space. Knowing this, how can we improve the algorithm from Question 1 so that it runs more efficiently (in time)?

¹A sentinel value in this context is any value which is certain not to be contained in the original array.

3. (*Bounded case*)

Suppose now that the array **arr** holds only integer values in $[0, k-1]$ for some positive integer k .

** a) By using an auxiliary array of booleans **contains** of size k , where **contains** $[i] = \mathbf{true}$ if i is in **arr**, and **false** otherwise, propose another new version of COUNTDISTINCTVALUES which runs in linear time $O(n+k)$.

Here you should note that k is an additional parameter of the problem, so the complexity should take both n and k into account.

** b) What is the space complexity of this new algorithm?

*** c) Compare this new algorithm to the general-case algorithm from Question 2(c). Which is better? Why?

Answer of exercise 2

1. (*General implementation*)

Procedure COUNTDISTINCTVALUES(**arr**)

$n \leftarrow \text{len}(\mathbf{arr})$, numDistinctVal $\leftarrow 0$;

for $i \leftarrow 0$ **to** $n-1$ **do**

```

    if  $\mathbf{arr}[i] \neq \mathbf{null}$ ;
    then
        numDistinctVal  $\leftarrow$  numDistinctVal + 1;
        REMOVE(arr,  $i$ );

```

▷ A new value is found

return numDistinctVal;

Procedure REMOVE(**arr**, i)

for $j \leftarrow i+1$ **to** $n-1$ **do**

```

    if  $\mathbf{arr}[j] = \mathbf{arr}[i]$  then
         $\mathbf{arr}[j] \leftarrow \mathbf{null}$ ;

```

The algorithm analysis is essentially the same as Exercise 1, Question 2. We have two nested loops, which gives quadratic time complexity, and constant space complexity.

2. (*Sorted case*) a) We provide the following algorithm:

Procedure COUNTDISTINCTVALUESSORTED(**arr**)

$n \leftarrow \text{len}(\mathbf{arr})$, numDistinctVal $\leftarrow 0$, $i \leftarrow 0$;

while $i < n$ **do**

```

    numDistinctVal  $\leftarrow$  numDistinctVal + 1;
    currentVal  $\leftarrow$   $\mathbf{arr}[i]$ ,  $j \leftarrow i+1$ ;
    while  $j < n$  and  $\mathbf{arr}[j] = \mathbf{arr}[i]$  do
         $j \leftarrow j+1$ ;
     $i \leftarrow j$ ;

```

return numDistinctVal;

b) The analysis is a little more complex, but we can see that the algorithm in fact performs a single “pass” through the array, since i “jumps” to the index of the next distinct value each time (which is equal to j at that point). This implies that the algorithm has linear time complexity. It has constant space complexity for the usual reasons.

c) In the general case, we can simply call `SORT(arr)` at the very start of the procedure, then call `COUNTDISTINCTVALUESORTED` on the sorted array. The time complexity will be the sum of the complexity of `SORT` and of `COUNTDISTINCTVALUESORTED`, i.e. $O(n \cdot \log(n) + n) = O(n \cdot \log(n))$, i.e. *log-linear* complexity. The space complexity remains constant, since both procedures have constant space complexity.

3. (*Bounded case*) a) We provide the following algorithm:

```

1 Procedure COUNTDISTINCTVALUESBOUNDED(arr)
2  $n \leftarrow \text{len}(\text{arr})$ ;
3 contains  $\leftarrow [\text{false}] * k$ ;      ▷ Initialise an array of length  $k$  with all elements false
4 for  $i \leftarrow 0$  to  $n - 1$  do
5   | contains[arr[i]]  $\leftarrow$  true;
6 numDistinctVal  $\leftarrow 0$ ;
7 for  $j \leftarrow 0$  to  $k - 1$  do
8   | if contains[j] = true then
9     | | numDistinctVal  $\leftarrow$  numDistinctVal + 1;
10 return numDistinctVal;

```

Note that we would normally just write “if **contains**[j]” as our condition on Line 8, omitting the (redundant) “= **true**”, but we have included it here for clarity.

b) It is clear that the algorithm runs in $O(n + k)$ time since it just runs two for loops of length n and k one after the other. In terms of space complexity, it requires an additional array of length k (the auxiliary array **contains**), so the complexity is $O(k)$.

c) This depends on k . The previous algorithm had log-linear time complexity (in n) and constant space complexity, while this new version has $O(n + k)$ time complexity and $O(k)$ space complexity. If k is small relative to n , this version is better, provided the relatively small amount of additional memory required is not a problem. On the other hand, if k is (much) larger than n , say of order n^2 , then the previous version is superior (even in time, since $O(n + k) = O(k) \gg O(n \cdot \log(n))$ in that case). If k is of order between n and $n \cdot \log(n)$, then the algorithms are roughly comparable. The previous version (general) may be slightly slower, but costs less in memory, while the bounded version may be slightly faster, but requires more memory overhead.

Exercise 3 Maximum sub-array sum

The goal of this exercise is to solve the following problem. Given an array of numbers (which may contain negative values), find the maximum sum of any *contiguous sub-array*. In other words, given a sequence of numbers $a_0, \dots, a_{n-1}$ (the elements of the array), find the maximum value of $\sum_{i=j}^k a_i$ where $0 \leq j \leq k < n$.

1. (*Brute-force approach*)

- * a) Write a helper procedure `PARTIALSUM(arr, j, k)` that calculates the partial sum of elements of `arr` from index j to index k included. What is the time complexity of this procedure?
- ** b) Using the `PARTIALSUM` procedure, propose a `MAXSUBARRAYSUM(arr)` algorithm that solves the problem by calculating and checking the value of the partial sums for every possibly pair (j, k) .²
- ** c) What are the time and space complexities of this algorithm?

2. (*Prefix approach*)

In general, brute-force approaches such as that of the previous question tend not to be very efficient in terms of time complexity.

- ** a) The previous algorithm contains a flaw in that it repeats calculations unnecessarily. Explain how/where this occurs.
- b) The idea is that our helper procedure `PARTIALSUM` satisfies the following recurrence relation: $\text{PARTIALSUM}(\text{arr}, j, k + 1) = \text{PARTIALSUM}(\text{arr}, j, k) + \text{arr}[k]$. As such, when we want to compute $\text{PARTIALSUM}(\text{arr}, j, k + 1)$, we should be able to do so in constant time, given we have already computed $\text{PARTIALSUM}(\text{arr}, j, k)$.
- *** i – Propose a refined version of the `MAXSUBARRAYSUM` that exploits this observation.
- * ii – What are the time and space complexities of this refined algorithm?

3. (*Optimal approach: Kadane's algorithm*)

In this question, we will give an optimal solution to the problem by using *Kadane's algorithm*. The key insight is as follows. For an array index k , let $\text{dp}(k)$ denote the maximum sub-array sum which ends at index k . Then we have:

$$\text{dp}(k) = \max \{ \text{arr}[k], \text{dp}(k - 1) + \text{arr}[k] \}.$$

In words, the max sum ending at a given position is either the array's value at that position (sum of sub-array of length 1) or the value at that position plus the max sum ending at the previous position.

- *** a) Using this observation, write a `MAXSUBARRAYSUM` algorithm that solves the problem using a single loop on the end-index k of sub-arrays. You will need a variable `current_max` tracking the *local* maximum $\text{dp}(k)$, as well as a variable

²Algorithms such as this one, which exhaustively check all possible solutions and return the “best” one, are known as a **brute-force** algorithms.

`global_max` tracking the *global* maximum (the max sum of all sub-arrays ending at or before index k).

- *** b) What are the time and space complexities of this new algorithm? The question states that this is the “optimal” approach, which means it can’t be improved on. Give an intuition for why that is the case.

Answer of exercise 3

1. (*Brute-force approach*)

Procedure PARTIALSUM(`arr`, j , k)

partialSum $\leftarrow$ 0;

for $i \leftarrow j$ **to** k **do**

 partialSum $\leftarrow$ partialSum + `arr`[i];

return partialSum

Procedure MAXSUBARRAYSUM(`arr`)

$n \leftarrow \text{len}(\text{arr})$, maxSum $\leftarrow -\infty$;

for $j \leftarrow 0$ **to** $n - 1$ **do**

for $k \leftarrow j$ **to** $n - 1$ **do**

 maxSum $\leftarrow$ max {maxSum, PARTIALSUM(`arr`, j , k)};

return maxSum;

We have three nested for loop, so the time complexity is $O(n^3)$, while the space complexity is still $O(1)$.

2. (*Prefix approach*)

- a) The flaw is essentially explained in the question. When we calculate PARTIALSUM(`arr`, j , $k+1$), we in fact repeat the calculations from PARTIALSUM(`arr`, j , k), making the computation inefficient.

- b) We can improve the algorithm by tracking the current sum in the k -loop, as follows.

Procedure MAXSUBARRAYSUM(`arr`)

$n \leftarrow \text{len}(\text{arr})$, maxSum $\leftarrow -\infty$;

for $j \leftarrow 0$ **to** $n - 1$ **do**

 currentSum $\leftarrow$ 0;

for $k \leftarrow j$ **to** $n - 1$ **do**

 currentSum $\leftarrow$ currentSum + `arr`[k];

 maxSum $\leftarrow$ max {maxSum, currentSum};

return maxSum;

The new algorithm has quadratic time complexity (two nested loops), and constant space complexity.

3. (Kadane's algorithm)

Procedure MAXSUBARRAYSUM(**arr**)

$n \leftarrow \text{len}(\text{arr})$, $\text{localMax} \leftarrow -\infty$, $\text{globalMax} \leftarrow -\infty$;

for $k \leftarrow 0$ **to** $n - 1$ **do**

$\text{localMax} \leftarrow \max \{ \text{arr}[k], \text{localMax} + \text{arr}[k] \};$ $\triangleright$ Tracks the value of $\text{dp}(k)$
 $\text{globalMax} \leftarrow \max \{ \text{localMax}, \text{globalMax} \}$

return globalMax ;

The algorithm clearly has linear time complexity and constant space complexity. To see that the time complexity is optimal, we note that it is necessary to retrieve all the values from the array. For example, if all the elements of the array are strictly positive, then the maximal sum is the sum of all the elements, and calculating the sum requires all the values of elements in the array to be retrieved. Since retrieving all the values from the array takes linear time, this implies that any algorithm which solves the problem will take at least linear time.

Exercise 4 Well-balanced expressions

Recall from the lecture our algorithm using a stack to check if a given expression is well-balanced or not. In this exercise, we will work a little more on this problem.

1. (Single-delimiter case)

In this question, we consider that the input expression **expr** only uses one type of delimiter/bracket, for example normal parentheses $()$.

- ** a)** For an index i of the **expr**, let $\text{open}(i)$, respectively $\text{close}(i)$, denote the number of opening, respectively closing, parentheses in **expr** at or before index i . For example, for the expression $((a + b) * ((c - d)/(e - f)))$, we have:

index i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
expr [i]	(	(	a	+	b	)	*	(	(	c	-	d	)	/	(	e	-	f	)	)	)
$\text{open}(i)$	1	2	2	2	2	2	3	4	4	4	4	4	4	4	5	5	5	5	5	5	5
$\text{close}(i)$	0	0	0	0	0	1	1	1	1	1	1	1	2	2	2	2	2	2	3	4	5

Now consider the quantity $\text{diff}(i) := \text{open}(i) - \text{close}(i)$. What properties must be satisfied by the sequence $(\text{diff}(i))_{0 \leq i < n}$ for the expression to be well-balanced?

- * b)** Using the characterisation with $\text{diff}(i)$ from the previous question, write an algorithm ISWELLBALANCED(**expr**) that returns **true** if **expr** is well-balanced, and **false** otherwise.
- * c)** What are the time and space complexities of this new algorithm? Why is it better, in this specific case, than the algorithm seen in the lecture?

2. (General case)

We now return to the general case where the expression may contain more than one type of delimiter. Start by recalling (without your notes, if possible), the algorithm from the lecture, which uses a stack.

- ** a) Why is a stack necessary? Why can't we just use a similar approach to the previous question, with separate counter variables for each delimiter type? Give an intuition for why this approach would not work, or a counter-example on which it fails.
- * b) Modify the algorithm `ISWELLBALANCED(expr)` from the lecture slides so that it returns the first *index* at which the well-balanced test fails, or -1 if the expression is well-balanced. For example, for the expression "`([])`", it should return 2, the index of '`)`'.

Answer of exercise 4

1. (*Single-delimiter case*)

- a) The sequence satisfies $\text{diff}(n - 1) = 0$ (the expression has equal numbers of opening and closing parentheses), and $\text{diff}(i) \geq 0$ for all $i \in [0, n - 1]$ (well-balanced property).
- b) Using the previous question, we can write:

```

Procedure ISWELLBALANCED(expr)
   $n \leftarrow \text{len}(\text{expr})$ ,  $\text{diff} \leftarrow 0$ ;
  for  $i \leftarrow 0$  to  $n - 1$  do
    if isOpeningDelimiter(expr[ $i$ ]) then
      |  $\text{diff} \leftarrow \text{diff} + 1$ ;
    else if isClosingDelimiter(expr[ $i$ ]) then
      |  $\text{diff} \leftarrow \text{diff} - 1$ ;
      | if  $\text{diff} < 0$  then
      | | return false;
  return  $\text{diff} == 0$ ;

```

- c) The algorithm has linear time complexity, which is the same as the algorithm seen in the lecture. However, it has constant space complexity, which improves the linear space complexity of the lecture algorithm (due to the auxiliary stack). This is not surprising: special cases can usually be solved more efficiently than the general case.
- #### 2. (*General case*)
- a) The issue is potential "interleaving" between different types of delimiters. For example, the expression "`[()]`" is not well-balanced, but if we run the algorithm from the previous question separately on both types of delimiters, it would return true.
 - b) This is a straightforward modification: we only need to change the return statements from Slide 11 of the lecture notes.

Procedure IsWELLBALANCED(*expr*)

$S \leftarrow \text{Stack}(), i \leftarrow 0;$

while $i < \text{len}(\textit{expr})$ **do**

$c \leftarrow \textit{expr}[i];$

if *isOpeningDelimiter*(c) **then**

$S.\text{push}(\textit{expr}[i]);$

else if *isClosingDelimiter*(c) **then**

if $S.\text{isEmpty}()$ **then**

return $i;$

else

$c' \leftarrow S.\text{pop}();$

if c and c' do not match **then**

return $i;$

$i \leftarrow i + 1;$

if $S.\text{isEmpty}()$ **then**

return -1

else

return $i;$

▷ return n in this case